

OpenCode + Oh My OpenCode

GitHub Copilot을 OpenCode에 연결하고, Oh My OpenCode로 확장하는 개발 워크플로 (이종훈)



PART 1

오프닝 — AI 코딩, 어디까지 왔나

GitHub Copilot의 진화로 보는 AI 코딩의 역사

GitHub Copilot의 진화로 보는 AI 코딩의 역사

2022~2023

Copilot v1

자동완성의 시대

Tab으로 코드 제안
Chat Completion: 프롬프트 →
응답 → 끝
실행 권한 없음



2024

Copilot Chat/Workspace

대화형으로 진화

IDE 내 질문/답변, 리팩터링 제안
여전히 '제안'까지만
Cursor, Windsurf 등장



2025~

Coding Agent

에이전트의 시대

파일 수정, 셸 실행, 빌드/테스트
GH 이슈 → PR 자동 생성
Agentic Loop 구조

Chat Completion vs CLI 에이전트

근본적 차이

Chat Completion

ChatGPT, Copilot Chat 등

- 프롬프트 → 응답 → 끝 (단발성)
- 코드를 '보여줄' 순 있지만 '실행' 불가
- 컨텍스트 = 대화 내용뿐
- 개발자가 복붙 → 오류 → 다시 질문

네비게이션이 길만 알려주고,
운전은 내가 하는 것

CLI 에이전트

OpenCode, Claude Code 등

- 계획 → 도구 호출 → 결과 관찰 → 반복
- 파일 읽기/쓰기, 빌드, 테스트 직접 수행
- 실패 시 스스로 에러 수정 (self-healing)
- 개발자는 '승인/거부'만 — 감독자 역할

자율주행 — 목적지만 말하면
알아서 간다

핵심 도구 비교

Chat Completion vs CLI 에이전트

	Chat Completion	CLI 에이전트
파일 읽기/쓰기	✗	✓
셸 명령 실행	✗	✓
빌드/테스트 실행	✗	✓
에러 자동 수정	✗	✓ self-healing
프로젝트 구조 인식	✗	✓ LSP, glob, grep
실행 루프	수동 (복붙)	자동 (agentic loop)
개발자 역할	실행자	감독자

오늘 이야기할 것



OpenCode

어떤 프로바이더에도 종속되지 않는
오픈소스 AI 에이전트



Oh My OpenCode

6명의 전문 에이전트가 협업하는
AI 개발팀 오케스트레이션 플러그인

"모델을 고르는 게 아니라, 팀을 편성하는 시대"

PART 2

OpenCode 소개

OpenCode란?

100% 오픈소스 AI 코딩 에이전트

120K+

GitHub Stars

800+

기여자

5M

월 활성 사용자

75+

LLM 프로바이더

멀티 플랫폼

터미널 TUI (Bubble Tea)
데스크톱 앱 (BETA)
IDE 확장 (개발 중)

클라이언트-서버

모바일에서 원격 조작
프론트엔드는 클라이언트 중 하나

프로바이더 자유

Anthropic, OpenAI, Google, Groq
AWS Bedrock, Azure, 로컬 모델
GitHub Copilot 토큰 지원

핵심 기능 빠르게 훑기



Build / Plan 에이전트

Tab 키 전환: Build(전체 권한)
↔ Plan(읽기 전용, 분석)



서브에이전트

@general — 멀티스텝 리서치
@explore — 빠른 코드 탐색



자동 LSP

Language Server 자동 감지
IDE 수준 코드 인텔리전스



커스텀 에이전트/커맨드

JSON/MD로 자유 정의
/review 같은 커맨드 생성



MCP 서버 지원

stdio/SSE 기반 MCP 연결
외부 도구 생태계 통합



권한 시스템

allow / ask / deny 세분화
bash 명령별 glob 패턴 매칭

PART 3

Oh My OpenCode (OmO)

게임 체인저 — 멀티모델 에이전트 오케스트레이션

OmO란 무엇인가?

OpenCode를 위한 멀티모델 에이전트 오케스트레이션 플러그인

"Claude Code가 1명의 시니어 개발자라면, OmO는 6명의 전문가로 구성된 AI 개발팀"

모델 특화

모델마다 잘하는 게 다르다
Claude: 오케스트레이션
GPT: 추론
Gemini: 프론트엔드

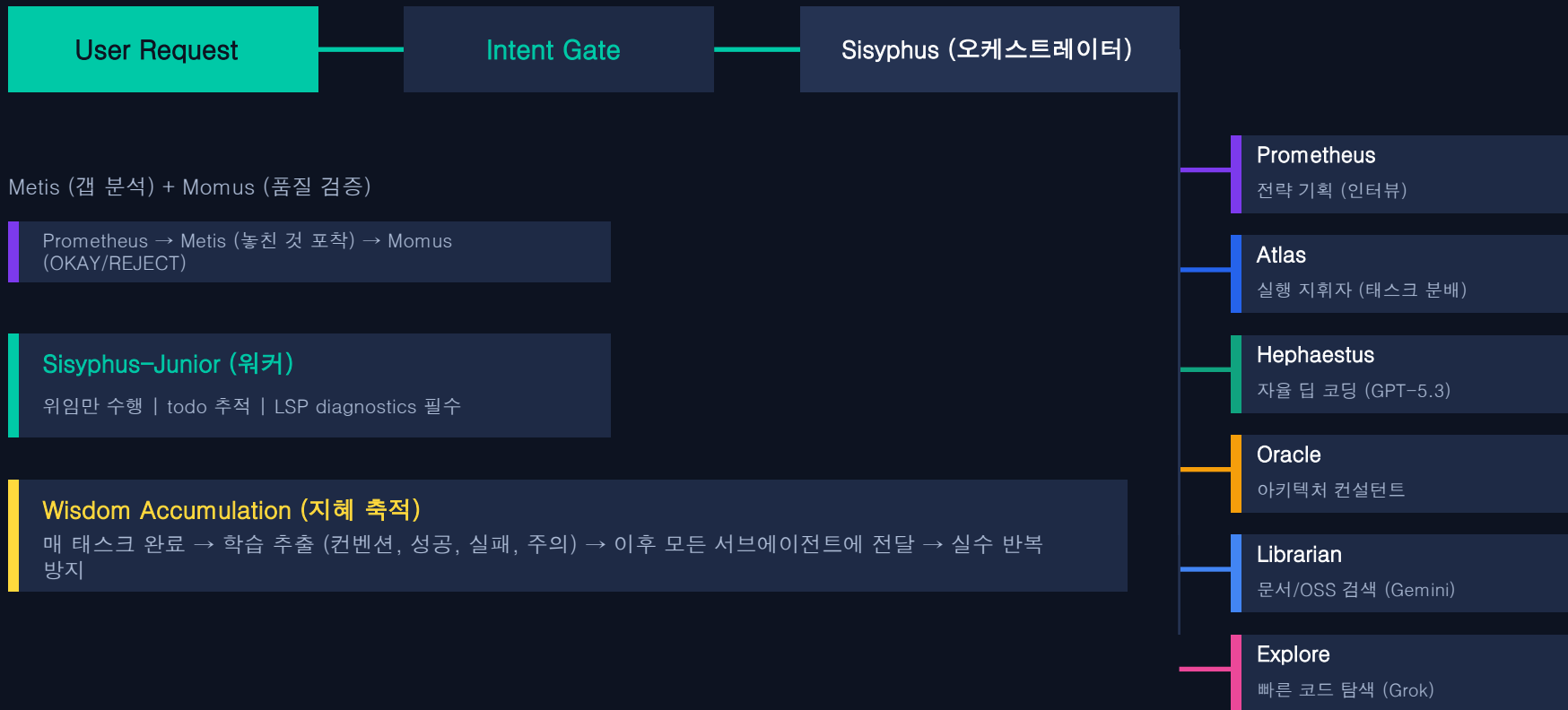
미래 대비

모델은 매달 싸지고 특화된다
단일 모델 종속은 미래가 없다
카테고리 기반 자동 라우팅

탄생 배경

Anthropic의 API 차단 사건
→ Hephaestus(GPT 기반) 탄생
"좋은 감옥이지만 여전히 감옥"

에이전트 아키텍처 딥다이브



카테고리 시스템

모델 이름이 아닌 의도로 라우팅

기존 방식 (모델 이름 직접 지정)

```
task({ agent: "gpt-5.4", ... })  
task({ agent: "claude-opus-4.6", ... })
```

OmO 방식 (시맨틱 카테고리)

```
task({ category: "ultrabrain", ... })  
task({ category: "quick", ... })
```

카테고리	기본 모델	용도
visual-engineering	Gemini 3.1 Pro	프론트엔드, UI/UX
ultrabrain	GPT-5.4 (xhigh)	깊은 논리, 아키텍처
deep	GPT-5.3	자율 탐색 + 딥 코딩
quick	GPT-5.4 Mini	사소한 변경, 오타 수정
unspecified-high	Claude Opus 4.6	복잡한 비즈니스 로직
writing	Gemini 3 Flash	문서, 기술 글쓰기

ultrawork (울트라워크)

한 마디로 시작하는 자율 개발

> ultrawork

입력 한 마디로 자율 개발 시작

1

코드베이스
탐색

패턴 리서치

2

기능 구현

병렬 에이전트

3

진단 검증

LSP + 테스트

4

완료까지 반복

멈추지 않음

추천 구독 조합

ChatGPT \$20 + Kimi \$0.99 + GLM \$10 = 약 \$31/월

Intent Gate + Ralph Loop + Todo Enforcer

실행 전 분류, 완료까지 반복

Intent Gate (의도 게이트)

- 리서치? → 검색 에이전트
- 구현? → 빌드 에이전트
- 조사? → 읽기 전용
- 수정? → 디버깅 플로우

Ralph Loop (자기 참조 루프)

- `/ulw-loop` 실행
- 100% 완료까지 무한 반복
- "다 했다" → 검증 → 미완료 → 재투입
- 시시포스의 바위처럼 멈추지 않음

Todo Enforcer — 시스템 리마인더

[SYSTEM REMINDER - TODO CONTINUATION]

미완료 **todo**가 있습니다! 전부 완료 전까지 응답하지 마세요:

- [] 유저 서비스 구현 ← 진행 중
- [] 유효성 검증 추가

실전 도입 가이드

설치부터 팀 워크플로우까지

1 OpenCode 설치

```
npm i -g opencode-ai && opencode
```

2 OmO 설치

에이전트에게 설치 지시 한 줄로 완료

팀 워크플로우에 녹이기

- AGENTS.md를 Git에 커밋 — 팀 전체 동일 AI 컨텍스트
- 커스텀 커맨드로 표준화: /test, /review, /deploy
- /share로 세션 링크 생성 — AI 대화 이력 협업
- oh-my-opencode.jsonc — 프로젝트별 모델/권한 설정

핵심 메시지

- 01 모델은 도구다 — 목적에 맞는 도구를 골라 쓰자
- 02 오케스트레이션이 핵심 — 개별 모델 성능보다 팀 편성이 중요
- 03 오픈소스가 답 — 벤더 락인 없이 최신 모델을 즉시 활용

**"AI 코딩의 미래는 단일 모델이 아닌,
최적의 팀 편성이다"**

```
$ npm i -g opencode-ai && opencode
```